CORRIGÉ · PROBLÈME · ANNALES SPÉ

Estimer le cardinal d'un flot : HyperLogLog

Énoncé

Cardinal d'un flot = nombre de valeurs distinctes. Le premier bit activé $\varrho(h)$ d'un entier $h \in [\![0, 2^{62}-1]\!]$ est le plus grand i tel que les $i-1$ bits de droite sont nuls ; $\varrho(0) = 63$ (exemples : $\varrho(1) = 1$, $\varrho(6) = 2$, $\varrho(8) = 4$). Hachage par `String.hash`.

1. Écrire `ffs (h:int) : int`.

Deux algorithmes sur les t premiers termes : `cardinality_nf` remplit une table de hachage et renvoie son nombre d'éléments ; `cardinality_fm` garde $r = \max \varrho(H(u))$ et renvoie 2^r.

1. Comparer leurs complexités en espace.
2. Écrire `harmonic_mean (z:float array) : float` ($\text{Harm} = m(\sum \frac{1}{z_j})^{-1}$).

HyperLogLog ($m = 16$, $\alpha_{16} = 0{,}6731$) : le terme u va dans le sous-flot $i = H(u) \bmod m$, et $r_i = \max \varrho(H(u)/m)$ sur ce sous-flot ; estimation $s \simeq \alpha_m \cdot m \cdot \text{Harm}(2^{r_0}, \dots, 2^{r_{m-1}})$.

1. Écrire `hll_update (r:int array) (x:string) : unit` (met à jour r avec le nouveau terme x).
2. Écrire `mean_estimate (r:int array) : float`.
3. Dans le code multi-fils donné (deux fils remplissent `r_u` et `r_v` par `hll_update` ; le fil principal calcule toutes les 5 secondes `Array.init m (fun i -> max r_u.(i) r_v.(i))` et affiche l'estimation), identifier les courses critiques et proposer un mécanisme de correction.

Décoder l'énoncé

Ce que l'énoncé te donne.

- $\varrho(h)$ est la position (à partir de 1) du bit 1 le plus à droite.

Ce qu'on te demande. Estimer le nombre de valeurs distinctes avec une mémoire minuscule, puis protéger un calcul partagé entre fils d'exécution.

Corrigé

Question 18

Étape 1. Q18. ffs

```
let ffs (h : int) : int =
  if h = 0 then 63                          (* convention *)
  else begin
    let rec aux h i = if h land 1 = 1 then i else aux (h lsr 1) (i + 1) in
    aux h 1
  end
```

Vérification : $6 = 110_{(2)}$: le bit de poids 1 est nul, le suivant vaut 1, donc $\varrho(6) = 2$.

Question 19

Étape 1. Q19. Mémoire des deux algorithmes

- `cardinality_nf` stocke toutes les chaînes distinctes dans la table : $O(s)$ chaînes, donc une mémoire proportionnelle au cardinal (multipliée par la longueur des chaînes).
- `cardinality_fm` ne garde qu'un entier $r \leqslant 63$: mémoire $O(1)$ (6 bits suffisent).

Le second est donc bien plus économe, au prix d'une estimation approchée (et grossière, puisqu'elle ne prend que des puissances de 2).

Question 20

Étape 1. Q20. Moyenne harmonique

```
let harmonic_mean (z : float array) : float =
  let m = Array.length z in
  let s = Array.fold_left (fun acc x -> acc +. 1. /. x) 0. z in
  float_of_int m /. s
```

Question 21

Étape 1. Q21. hll_update

```
let hll_update (r : int array) (x : string) : unit =
  let h = String.hash x in
  let i = h mod m in                        (* numéro du sous-flot *)
  r.(i) <- max r.(i) (ffs (h / m))          (* nouveau maximum de rho sur ce sous-flot *)
```

Seule la case i change : elle passe de $\max_{u \in U_{t-1}^i}$ à $\max_{u \in U_t^i}$, comme demandé.

Question 22

Étape 1. Q22. mean_estimate

```
let mean_estimate (r : int array) : float =
  let z = Array.map (fun ri -> 2. ** float_of_int ri) r in   (* les 2^{r_i} *)
  alpha16 *. float_of_int m *. harmonic_mean z
```

On calcule 2^{r_i} en flottant (`1 lsl 63` déborderait pour $r_i = 63$).

Question 23

Étape 1. Q23. Courses critiques

Accès concurrents : le fil `f1` écrit dans `r_u` (par `hll_update`, qui lit `r.(i)` puis l'écrit), pendant que le fil principal lit `r_u` à la ligne 46 ; de même pour `f2` et `r_v`. Les portions critiques sont donc les appels `hll_update r_u` (ligne 38) et `hll_update r_v` (ligne 42), et la construction de `r` (ligne 46). Le fil principal peut lire un tableau en cours de mise à jour et afficher une estimation incohérente.

Correction : un verrou par tableau (`Mutex`). On encadre chaque mise à jour et la lecture :

```
let lock_u = Mutex.create () and lock_v = Mutex.create () in
let f1 () = iter (fun x -> Mutex.lock lock_u; hll_update r_u x; Mutex.unlock lock_u)
              Int.max_int u in
(* idem pour f2 avec lock_v *)
...
Mutex.lock lock_u; Mutex.lock lock_v;              (* toujours dans le même ordre *)
let r = Array.init m (fun i -> max r_u.(i) r_v.(i)) in
Mutex.unlock lock_v; Mutex.unlock lock_u;
print_float (mean_estimate r)
```

On prend les verrous toujours dans le même ordre pour éviter tout interblocage, et on les relâche avant l'affichage et la pause.

Le piège

En Q21, utiliser le même nombre h pour choisir le sous-flot et calculer ϱ : les bits de poids faible seraient corrélés au numéro du sous-flot. D'où le quotient h/m.

La méthode

Estimation probabiliste : un résumé de taille fixe (maximum de ϱ) et une moyenne (harmonique) sur plusieurs sous-flots pour réduire la variance.

Rappels de cours utilisés

- **Verrous (Mutex).** Une section critique accède à une donnée partagée entre fils d'exécution. On l'encadre par `Mutex.lock` et `Mutex.unlock` d'un même verrou : un seul fil à la fois peut l'exécuter.